294_Projektdokumentation

Fabian Imhasly

Modul 295

Inhalt

Grundidee 2
Anforderungskatalog 2
Klassendiagramm 2
Storyboard 3
User Stories 3
User Story 1 3
User Story 2 3
User Story 3 4
Testplan 4
Testergebnisse 5
Installationsanleitung: 5
Voraussetzungen 5
Hilfestellungen 6

Grundidee

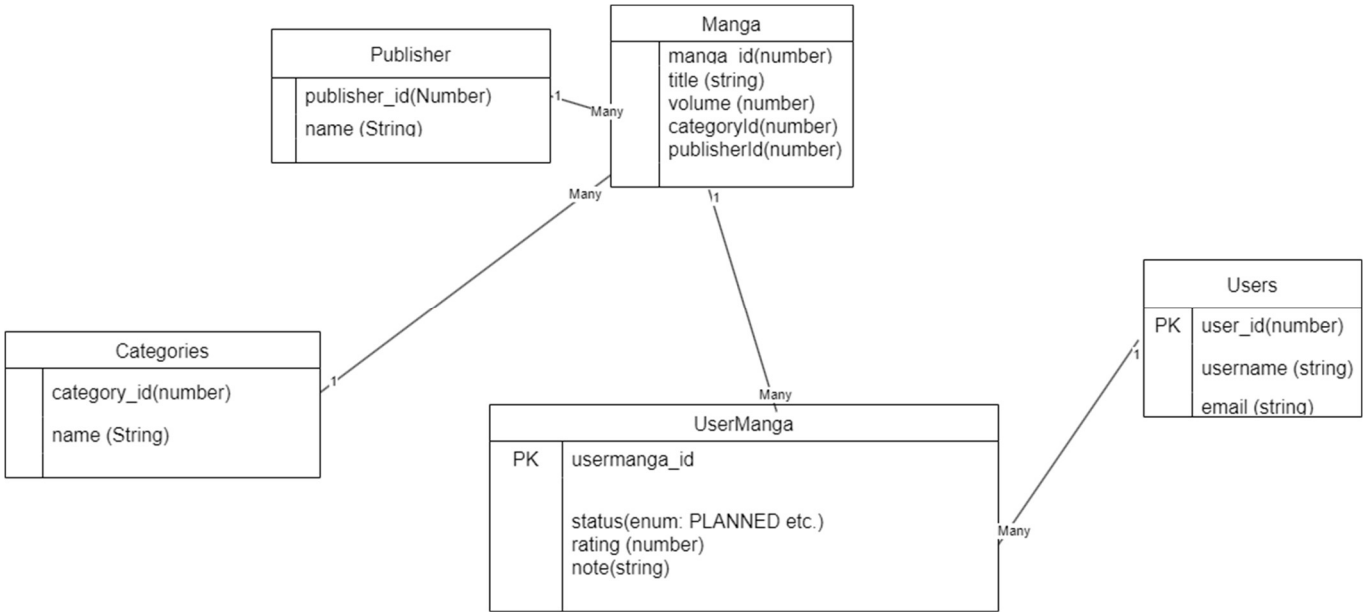
In meinem Projekt entwickle ich eine Web-Applikation „Manga Tracker“. Die Applikation unterstützt den Prozess, eine persönliche Manga-Bibliothek zu verwalten und den Lesestatus pro Manga einzutragen. Jeder Manga gehört zu genau einer Serie, wodurch Serien übersichtlich dargestellt werden können. Der Benutzer kann neue Serien und Mangas erfassen, bestehende Einträge bearbeiten sowie nicht mehr benötigte Einträge löschen. Zusätzlich kann der Status eines Mangas (z. B. „READING“, „COMPLETED“, „PLANNED“) im Frontend geändert werden. Die Daten werden über eine REST-API geladen und gespeichert.

Anforderungskatalog

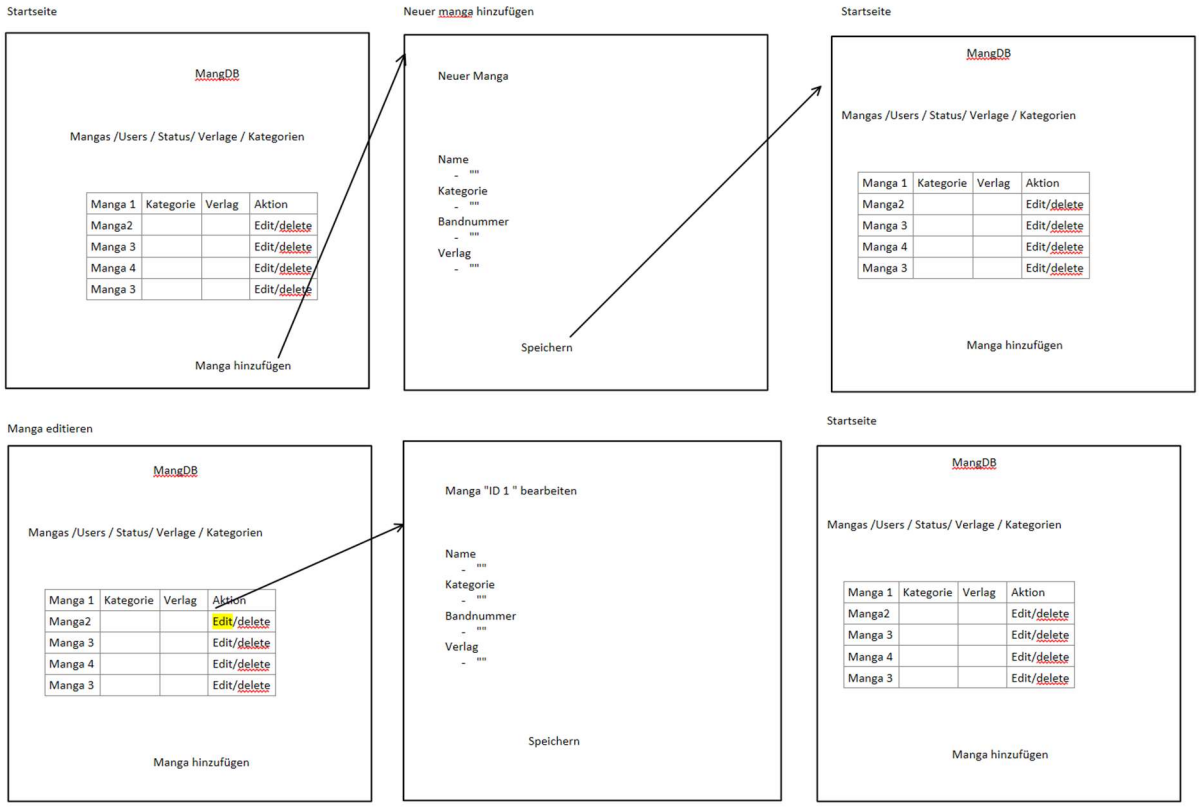
- Serien anzeigen (Liste)

- Anzeige aller Mangas aus dem Backend
- Erfassen neuer Mangas über ein Formular
- Bearbeiten bestehender Mangas
- Löschen bestehender Mangas
- Setzen und Ändern des Manga-Status
- Navigation über Routing (mehrere Views)
- Clientseitige Validierung von Pflichtfeldern
- Fehlerhandling bei REST-API-Fehlern
- Kommunikation mit REST-API über http

Klassendiagramm



Storyboard



User Stories

User Story 1

Manga erfassen

Als Benutzer möchte ich einen Manga erfassen damit er in meiner Bibliothek erscheint.
Akzeptanzkriterien

- Pflichtfelder: titel, bandNr, serieId, status müssen gesetzt sein
- Bei Erfolg wird der neue Manga in der Liste angezeigt (ohne manuelles Reload)

User Story 2

Manga Status ändern

Als Benutzer möchte ich den Status eines Mangas ändern

Akzeptanzkriterien

- Status ist nur aus { READING,COMPLETED,PLANNED,DROPPED) auszuwählen
- UI zeigt danach neuen Status an
- Fehler vom API wird sichtbar dargestellt (z. B. Meldung oberhalb Formular)

User Story 3

Manga löschen

Als Benutzer möchte ich einen Manga aus der Liste Entfernen.

Akzeptanzkriterien

- Löschen erfordert eine Bestätigung
- Nach erfolgreichem Löschen ist der Manga nicht mehr in der Liste
- Falls API Delete fehlschlägt: Meldung anzeigen, Eintrag bleibt sichtbar

Testplan

Ziel:
Überprüfung der wichtigsten Funktionen der Frontend-Applikation (Anzeige, Erfassen, Bearbeiten, Löschen, Status setzen).

Voraussetzungen:

- MySQL läuft (Docker)
- Backend läuft (Spring Boot)
- API erreichbar unter http://localhost:8080
- Frontend läuft unter http://localhost:5173
- Tests werden im Browser durchgeführt

Umfang:

- Anzeige von Daten aus REST-API
- Neue Daten Hinzufügen
- Daten Editieren
- Löschen bestehender Daten
- Validierung und Fehlerhandling

Testergebnisse

Testfall-ID	Funktion	Beschreibung	Erwartung	Resultat
TP-01	**Mangas anzeigen**	**Seite „Mangas“ öffnen**	**Liste wird geladen und angezeigt**	**OK**
TP-02	**Manga erstellen**	**Formular ausfüllen und speichern**	**Eintrag erscheint in Liste**	**OK**
TP-03	**Validierung**	**Pflichtfeld leer lassen**	**Fehlermeldung, kein Speichern**	**OK**
TP-04	**Manga bearbeiten**	**Eintrag ändern und speichern**	**Änderungen sichtbar**	**OK**
TP-05	**Manga löschen**	**Löschen bestätigen**	**Eintrag entfernt**	**OK**

Unit tests

Sind unter 294_LB\manga-frontend\src\pages\mangas

Installationsanleitung:

Voraussetzungen

- Node.js
- npm
- Docker

Projekt beziehen:

ZIP-Datei herunterladen, entpacken und lokal speichern.

Datenbank starten:

Im Projektverzeichnis Datenbank mittels Docker starten « docker compose up -d»

Backend starten:

Das Spring-Boot-Backend kann manuell über den Befehl «mvn spring-boot:run» oder direkt über Eclipse durch das Ausführen des Maven-Projekts gestartet werden.

Frontend starten:

In den Frontend-Ordner wechseln, Abhängigkeiten mit npm install installieren und die App mit npm run dev starten.

Funktionstest:

Nach dem start sollte es unter http://localhost:5173 erreichbar sein.

Hilfestellungen

Für die Umsetzung des Projekts wurden folgende Hilfestellungen genutzt:

- Unterrichtsmaterialien
- Offizielle React-Dokumentation
- W3 Schools
- MDN Web Docs
- ChatGPT für Erklärungen, Beispielcode und Debuggen